COMP2221 Systems Programming Summative Coursework Report

[CWMP89]

1 Summary of Approach & Solution Design

1.1 Structure

The allocator was designed to manage a single contiguous heap region while prioritising robustness against environmental hazards such as radiation-induced bit flips and partial writes caused by brownouts. The main architectural approach stores all allocator metadata directly within the heap using strictly aligned 40-byte blocks. This alignment corresponds exactly to the physical size of the dual-metadata headers, ensuring that all payloads remain strictly word-aligned for performance and correctness.

Each block is bookended by a header and a footer to support bidirectional heap traversal. To guarantee integrity, both the header and footer store mirrored metadata units. Each unit contains a magic value for block identification, inverted copies of size and flag fields to detect uniform bit errors, and a calculated checksum, as can be seen below.

MetaUnit Structure

```c
typedef struct {
    uint32_t magic;
    uint32_t magic_inv;
    uint32_t size_and_flags;
    uint32_t size_inv;
    uint32_t checksum;
} MetaUnit;
```

BlockHeader Structure

```c
typedef struct {
    MetaUnit primary;
    MetaUnit mirror;
} BlockHeader;
```

Crucially, the corruption detection mechanism relies on the **FNV-1a hash algorithm** [1]. This specific variant was selected for its low computational overhead, allowing the allocator to reliably validate the integrity of every metadata field before attempting any operation. The footer structure follows the same logic as the header, providing a redundant recovery point if the primary header is damaged.

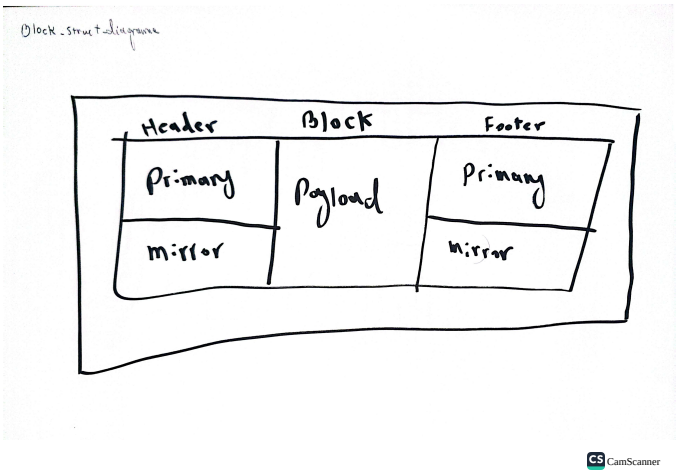


Fig. 1: Block structure: header, payload, footer, and metadata layout.

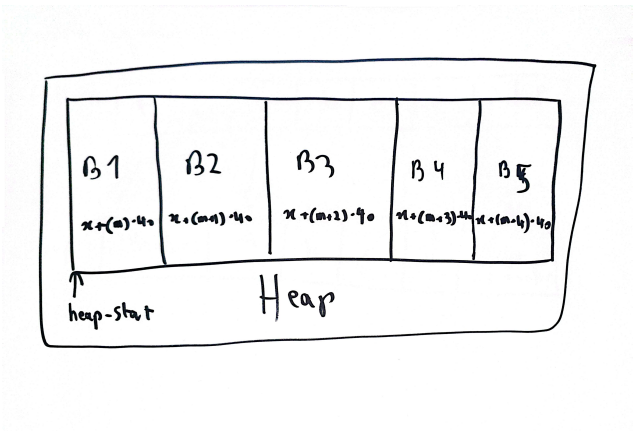


Fig. 2: Heap memory layout demonstrating 40-byte block alignment.

2 Analysis of Solution

2.1 Algorithm Implementation

The allocation strategy follows a linear scan of the heap in **mm_malloc()**. To ensure safety during traversal, every header is validated using **header_valid_only()**. If corruption is detected, the allocator attempts to recover by resynchro-

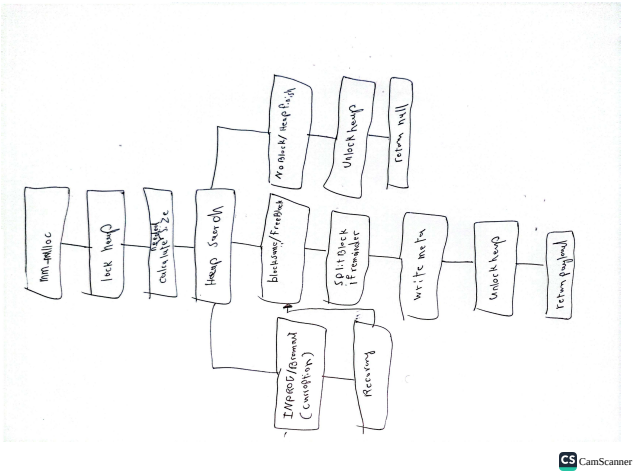


Fig. 3: Control flow and heap traversal during `mm_malloc`.

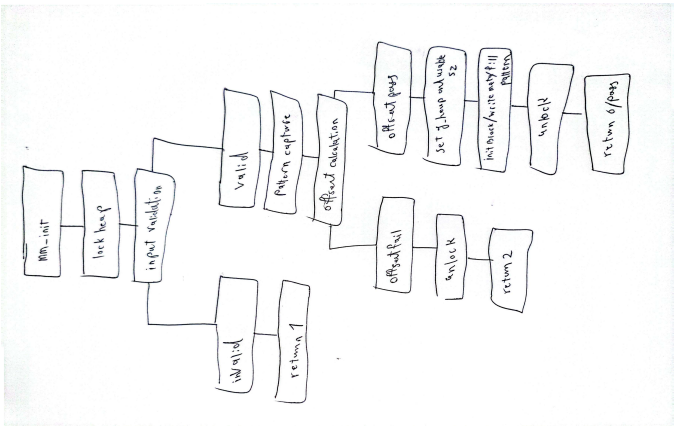


Fig. 4: Heap initialisation steps in `mm_init`.

nising forward to the next valid block boundary. Free blocks are coalesced to reduce fragmentation.

Initialisation is performed in **mm_init()**, which establishes the base heap alignment and captures the unique heap pattern used for brownout detection.

2.2 Trade-off Analysis

The allocator makes trade-offs between performance and memory overhead. Every block contains both a header and a footer, each storing primary and mirrored metadata, which increases reliability as well as metadata footprint. Every metadata unit includes checksums, inverted fields, and magic constantsas seen in the struct figures. The enforced 40-byte alignment (**MM_ALIGN**) also introduces internal fragmentation.

Performance is mainly affected by the allocator's linear heap traversal during allocation in **mm_malloc()**, which can degrade under fragmentation (see Table 2). Block validation (**block_sane()**) adds additional cost by verifying both metadata copies and computing checksums, particularly when blocks are allocated and payload checksums need to be recomputed.

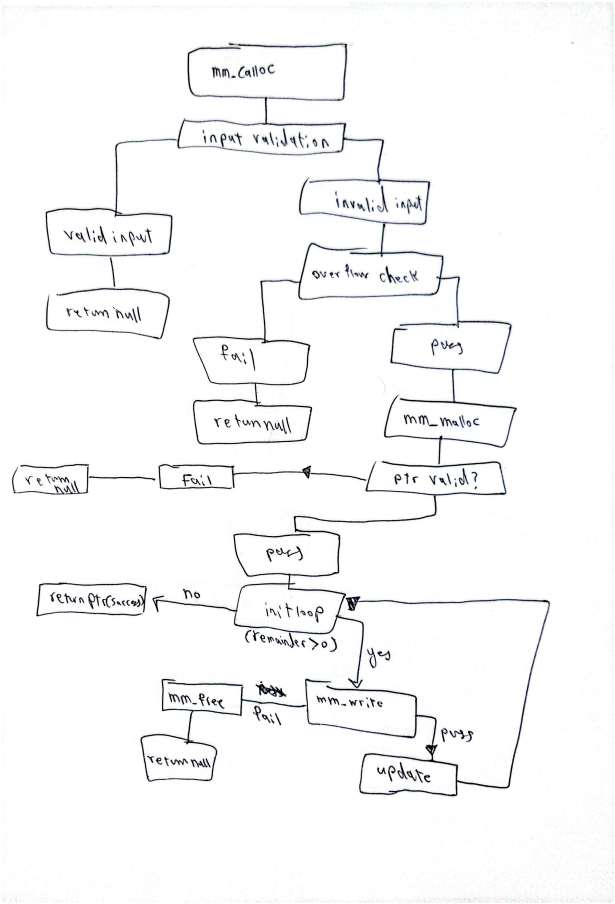


Fig. 5: Zero-initialisation and write-path interaction in `mm_calloc`.

TABLE 1: Payload Efficiency Analysis: Ratio of user-requested payload to actual block size occupied.

Request (B)	Block (B)	Overhead (B)	Eff. (%)
8	200	192	4.0
16	200	184	8.0
32	200	168	16.0
64	240	176	26.7
128	320	192	40.0
256	440	184	58.2
512	680	168	75.3
1024	1200	176	85.3
4096	4280	184	95.7

A major robustness advantage comes from the use of the **INPROG** flag for brownout safety. Metadata updates occur in stages within **write_block_meta()**, where mirror copies are marked as in-progress before committing a stable version to the primary, ensuring that partial writes don't leave hidden inconsistencies. The allocator uses pattern checks during reads in **mm_read()** to detect partial writes or corruption in regions that should contain user data or free-block patterns.

Overall, the implementation prioritises safety and recoverability over raw performance, which, in my opinion, is more appropriate in embedded or radiation-prone environments(at least in this case).

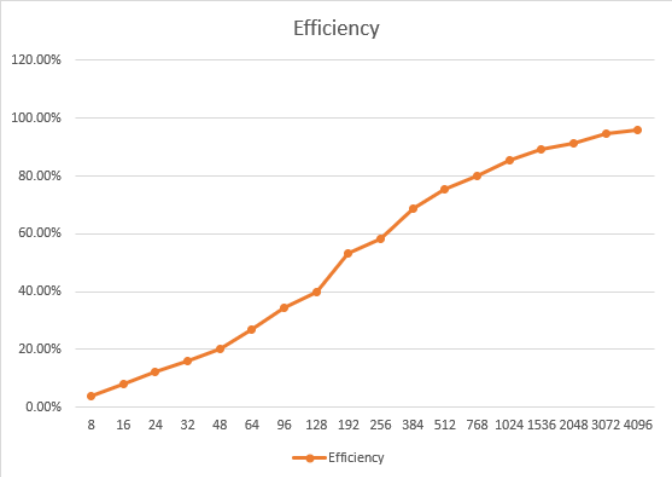


Fig. 6: Efficiency curve showing asymptotic improvement as block size increases. This figure visualises the data reported in Table 1.

TABLE 2: Latency Analysis: Comparison of allocation (linear search) vs. deallocation (constant time) costs.

Size (B)	Count	Alloc (μs)	Free (μs)
32	5242	601.607	0.414
64	4369	774.703	0.556
128	3276	1033.087	1.003
256	2383	1271.888	1.810
512	1542	1485.488	3.070
1024	873	1690.593	6.295

The newer validation helpers—mainly **ptr_to_header()** and enhancements inside **mm_heap_validate()** ensure pointer validity and full block consistency, reducing the chance of allocator misuse or silent corruption.

3 Use of Generative AI, Tools, or other Resources

The biggest use of Gen-AI was in the benchmarking of the program to help with fast tracking and bug fixes(only to help write and debug the benchmark code. All Tables seen are human generated and then drawn/plotted in Excel.). Other uses of this technology were mainly for explaining concepts and assisting in debugging the allocator. Additional usage involved formatting the code and test-polishing. The provided test runme.c was useful for validation early in the process, but later, only the autograder was used. No external allocator templates were used.

4 Additional functionality

Outside the main specifications, this implementation includes a few optional safety and diagnostic elements. The first is a quarantine mechanism used during **mm_free()**, where metadata that fails validation triggers and marks the block with the **BAD** flag instead of returning it to the free pool. Isolating corrupted regions and preventing allocator instability.

Another enhancement is the allocator's ability to recover from interrupted writes. If a block is found in the

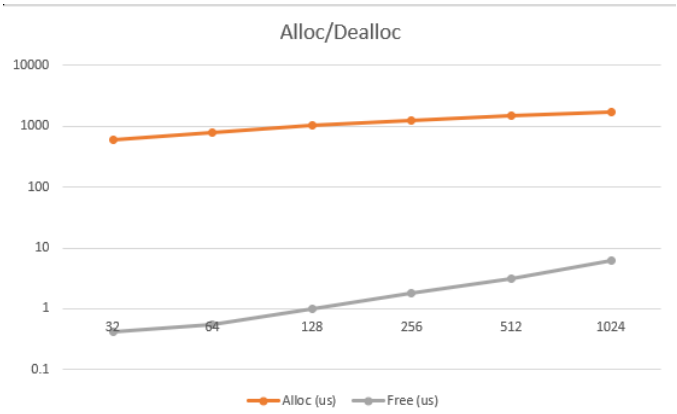


Fig. 7: Latency comparison chart (Logarithmic Scale) highlighting the performance gap between linear allocation and constant-time deallocation. This plot corresponds to the measurements in Table 2.

TABLE 3: I/O Throughput vs Hardware Baseline.

Size (B)	Read (MB/s)	Write (MB/s)	Memcpy (MB/s)
32	167.6	104.3	29785
64	187.5	121.3	64642
128	198.9	122.1	90062
256	229.7	135.2	111216
512	239.3	147.5	109983
1024	225.9	144.1	137069
4096	240.1	147.0	147857

INPROG state during **mm_read()** or **mm_free()**, it is safely deallocated and coalesced, ensuring that partially written or corrupted payloads do not persist. This behaviour improves reliability in unstable power environments.

The allocator comes with a comprehensive validation tool, **mm_heap_validate()**, which checks block consistency, footer alignment, **INPROG** states, free-block coalescing invariants, and free-block pattern correctness, which provides us with strong diagnostic value for development and runtime monitoring.

Finally, internal validation helpers such as **header_valid_only()**, **meta_pick()**, and **ptr_to_header()** were added to support improved consistency checking and safer pointer handling throughout the allocator. These do not change user-facing behaviour but increase internal resilience.

The implementation also provides a safe **mm_realloc()** wrapper which prioritizes heap stability. It validates the original pointer using **ptr_to_header()** to ensure it is not a corrupted or in-progress block. If valid, it allocates a completely new block of the requested size, copies the payload using **memcpy**, and then frees the old block. This minimizes the risk of metadata corruption during resizing operations.

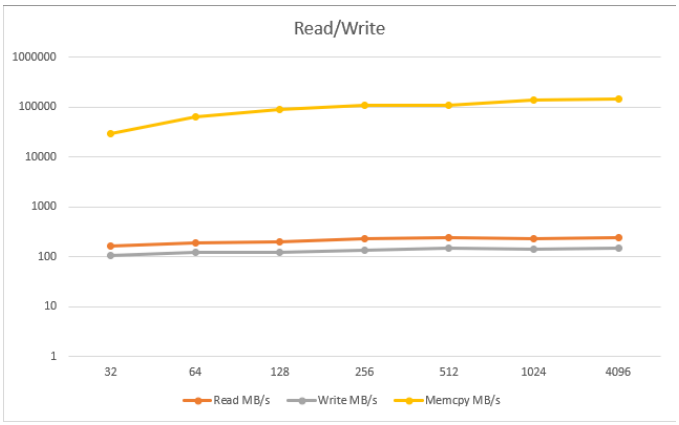
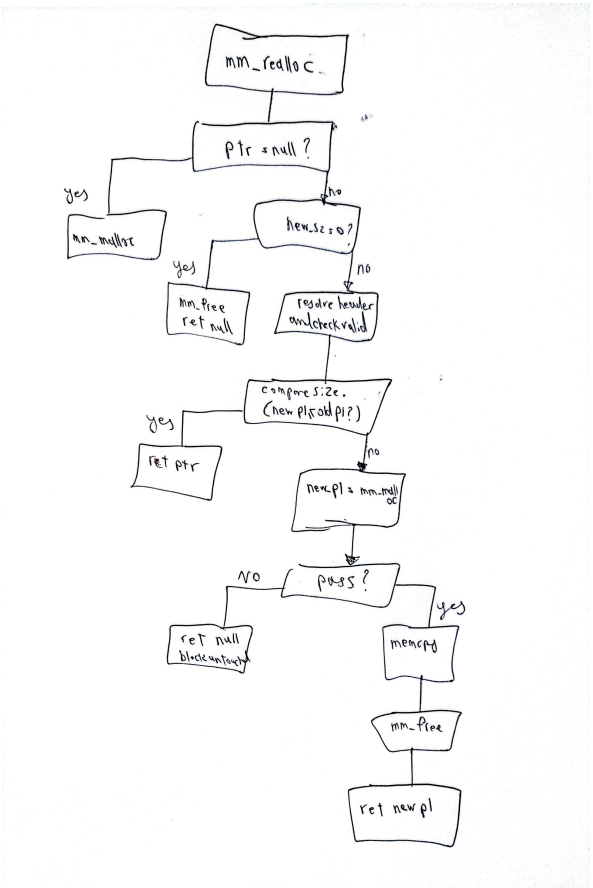


Fig. 8: Throughput comparison chart (Logarithmic Scale) showing the overhead of radiation-hardened I/O operations versus raw memory copy. This visualisation corresponds directly to Table 3.

TABLE 4: Sustained Workload Comparison: Performance and resilience under normal vs. hostile conditions (1 Million cycles).

Metric	Orbital (Stable)	Surface (Hostile)
Total Cycles	1000000	1000000
Env. Hazards	0 Radiation / 0 Power Surges	94 Radiation / 23 Power Surges
Operations		
Deploy (Alloc)	301159 (Failed: 0)	301623 (Failed: 0)
Recall (Free)	300139	300350
Telemetry (Read)	150249	149541
Update (Write)	149833	150231
Resilience		
Corruptions Caught	0	491
Final Heap State	MISMATCH	MISMATCH



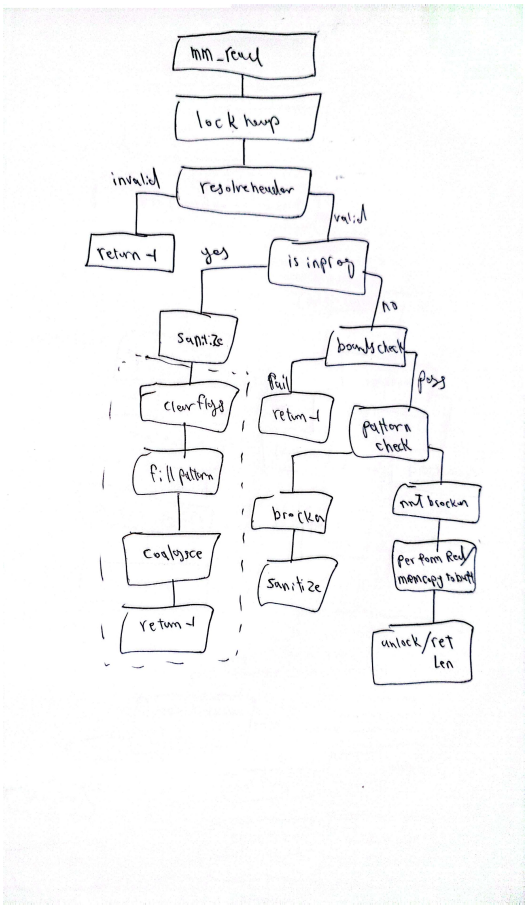


Fig. 9: Read path and corruption detection in `mm_read`.

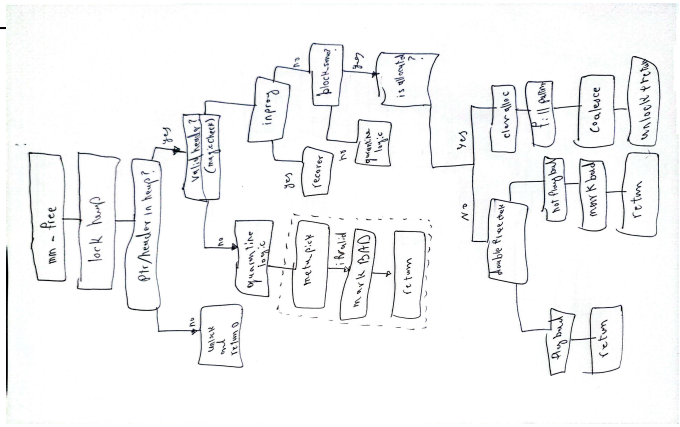


Fig. 10: Freeing, coalescing, and quarantine flow in `mm_free`.

Fig. 11: Safe reallocation strategy in `mm_realloc`.

REFERENCES

[1] G. Fowler, L. C. Noll, K.-P. Vo, D. E. E. 3rd, and T. Hansen, "The fnv non-cryptographic hash algorithm," Internet-Draft draft-eastlake-fnv-03, 2011, work in Progress. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-eastlake-fnv-03